

WHAT THIS SESSION IS

Run on 29 August 2026, and it is the widest session so far. It opens by breaking the dependency on one paid provider: closed models against open weights, a model running on your own laptop through Ollama, and a model running on Hugging Face's servers. Then it fills in the LangChain building blocks that everything later is assembled from: reading an image, prompt templates, output parsers, what a Runnable actually is, and finally LCEL, the pipe operator that turns three manual steps into one line. Six working examples, all in the same uv project from the previous class.

CONTENTS

1. Where this session picks up
2. Hugging Face, and what it is a repository of
3. Closed models against open weights
4. Running a model on your own laptop with Ollama
5. Reading an image, from a URL and from your disk
6. Prompt templates, and why the user should not write the prompt
7. Output parsers
8. Runnable: the common interface
9. LCEL, the pipe operator
10. Chaining a chain into another chain
11. A model from Hugging Face
12. Temperature, explained properly
13. Licences, cost and production reality
14. Clarifications from the Q&A
15. Key takeaways

1 Where this session picks up

The previous class built four small applications: a raw REST call, the OpenAI SDK, a first LangChain call, and messages with a system prompt and hand-rolled memory. Same uv project, same `.env`, so nothing is set up again here. New files are simply added alongside the old ones.

The plan announced at the start was: image input, Hugging Face, local models and open weights, a few remaining building blocks including structured output, and tool calling if time allowed. Everything except structured output and tool calling was covered, and those moved to the next day.

ON THE PACE, SAID PLAINLY

Feedback after the previous weekend split in two directions. One group found it too advanced and asked for more basics. Another found it too basic and asked to move faster. The stated approach is to keep everyone on the same track for the first few sessions, then accelerate. If a section here feels obvious to you, it is not aimed at you; the next one probably is.

2 Hugging Face, and what it is a repository of

Every application so far has used one trained model from one company. The question opening this section was simple: does it have to be that way?

The analogy given, and it is a good one, is GitHub. GitHub is where code lives: public repositories, private repositories, things built by a community, all discoverable in one place. Docker Hub is the same idea for container images.

Hugging Face is that place for trained AI models. Text generation, text summarisation, translation, image generation, speech recognition, embeddings. Hundreds of pages of them.

GitHub	→	source code, by a community
Docker Hub	→	container images
HUGGING FACE	→	trained AI models

so when you need a model, you have three routes:

- | | |
|------------------------------------|---------------------------------|
| 1. train your own | realistic for almost nobody |
| 2. pay a provider for a hosted one | OpenAI, Anthropic, Google |
| 3. take an existing open model | Hugging Face, or run it locally |

Route three splits again, and this distinction caused most of the questions in the session:

- **Download it and run it on your own machine.** That is what Ollama does, in section 4.
- **Leave it where it is and call it over the network.** The model runs on Hugging Face's infrastructure, not yours. That is section 11.

Official model index: huggingface.co/models

3 Closed models against open weights

This was the longest explanation of the evening and it is the one worth reading twice, because it is a decision you will make on every real project.

What a closed model actually means

When your code calls `gpt-4o-mini`, the question leaves your machine, travels to OpenAI's data centre, gets processed on their hardware, and the answer comes back. You have no visibility into or control over any of that. They charge per token because the GPUs doing the predicting are expensive and running constantly.

The reason it costs anything at all connects back to the first class. A model does not look answers up in a database, it predicts. Prediction means computation, and computation means hardware working continuously. The comparison used was your own brain: recognition only happens while you are awake and processing.

The two problems that follow

- **Cost.** Fine at twenty tokens for a demo. Your application has thousands of users making many calls each, and every one of them carries the whole conversation history. The running cost of the application climbs with usage in a way traditional software does not.
- **Privacy.** Your data goes to somebody else's servers. Every major provider offers enterprise agreements, private deployments and strong assurances. The honest question asked in class was whether you are willing to take that risk, and for a banking, healthcare or research product the answer is often no.

The alternative

Many companies publish the trained model itself. You download it and run it on your own machine or your own server. Now the prediction happens on your CPU, your RAM and your GPU, and your data never leaves.

CLOSED MODEL

your application

```
|  
| question + API key  
v
```

+-----+

```
| OpenAI / Google |  
| Anthropic ...   |  
|                 |  
| their GPUs      |  
| their servers   |  
| their control   |
```

+-----+

```
|  
v  
the answer
```

you pay per token

your data leaves

no infrastructure to run

best available quality

OPEN WEIGHTS, RUN LOCALLY

your application

```
|  
| question  
v
```

+-----+

```
| YOUR machine    |  
| YOUR RAM, GPU   |  
|                 |  
| the model file  |  
| sits on disk    |  
| your control    |
```

+-----+

```
|  
v  
the answer
```

you pay for hardware, once

your data stays

you run the infrastructure

usually a step behind

OPEN WEIGHTS, NOT OPEN SOURCE. THE DISTINCTION MATTERS.

In software, open source means you get the code and may modify it. Linux is open source, which is why Ubuntu and Amazon Linux exist.

With these models you get none of that. You do not get the training code, you do not get the training data, and you cannot see how it was trained. What you get is the trained weights, the file you download and run. That is why the correct term is **open weights**, and using "open source model" is, as put in class, the wrong term.

	Closed model	Open weights
Runs on	The provider's servers	Your machine or your server
You pay for	Tokens, per call	Hardware and electricity
Your data	Leaves your network	Stays where it is
API key	Required	Not needed for a local run
Can you download it	No	Yes
Can you fine tune it	Usually not, or only within limits	Yes
Control	Theirs	Yours
Quality	Generally the frontier	Generally a step behind, and improving
Internet needed	Yes	No, once downloaded

HOW TO CHOOSE, IN ONE LINE EACH

Security and privacy matter, token cost is a concern, and you can invest in infrastructure: **open weights**.
 Privacy is not the main worry, you cannot spend upfront on hardware, and you want the best answers: **closed model**.

The course uses closed models so that everybody can practise without buying a GPU. The reassurance given was that the code barely changes: swapping `ChatOpenAI` for `ChatOllama` leaves the rest of your application alone. If you join a company that only permits local models, what you learned still applies.

4 Running a model on your own laptop with Ollama

Ollama is the tool that runs an open weights model on your own machine. Install it, pull a model once, and it is available locally.

```
# one time
# install from https://ollama.com/download
ollama pull mistral
ollama list          # confirm it is there and running
```

Then add the LangChain integration to the project:

```
uv add langchain-ollama
```

ollama-demo.py

```
from langchain_ollama import ChatOllama

model = ChatOllama(
    model="mistral:latest",
    temperature=0
)

print(
    model.invoke(
        "What is LangChain in one line?"
    ).content
)
```

Compare that with the OpenAI version from the previous class. The only line that changed is the one that builds the model. `invoke`, `.content`, everything else, is identical. That is the payoff of working through a framework.

NOTICE THE TWO THINGS THAT ARE ABSENT

There is no `load_dotenv()` and no API key, because there is nothing to authenticate against and nothing to pay for. When this file runs, the prediction is done by your own RAM, processor and GPU. Asked directly in class, the answer was that this reply came from Mistral running on the laptop, not from OpenAI's servers.

PRACTICAL NOTES ON RUNNING MODELS LOCALLY

Ollama does not sit there consuming your machine when no model is loaded, and you can remove a model once you are done experimenting. Asked about hardware, the instructor's own laptop has 24 GB of RAM, which is comfortable for a small model.

Asked why Mistral specifically: it is small, it runs locally without much fuss, it costs nothing, and there is no vendor lock-in. For a class demonstration it is exactly right. It is trained on fewer parameters than a frontier model, so do not judge open weights as a category by it.

Model library: ollama.com/library

5 Reading an image, from a URL and from your disk

Back to OpenAI for this one, because it needs a vision capable model. The idea is that a `HumanMessage` does not have to carry a plain string. Its content can be a list of blocks, and each block declares what it is carrying.

a normal message

```
HumanMessage(  
    content="Describe a dog"  
)
```

a multimodal message

```
HumanMessage(content=[  
    { "type": "text",  
      "text": "What animal is this?" },  
    { "type": "image",  
      "url": "https:// ..." },  
)
```

two blocks, one message.

the same shape works across

any provider that supports vision.

Option one: an image already on the internet

```
from dotenv import load_dotenv
from langchain_openai import ChatOpenAI
from langchain_core.messages import HumanMessage

load_dotenv()

model = ChatOpenAI(model="gpt-4o-mini")

image_url = "https://raw.githubusercontent.com/pytorch/hub/master/images/dog.jpg"

url_message = HumanMessage(content=[
    {
        "type": "text",
        "text": "Describe this picture in two sentences. Also talk about the breed of
this."
    },
    {
        "type": "image",
        "url": image_url
    },
])

print(model.invoke([url_message]).content)
```

Run live, it came back with a description of a fluffy white dog sitting on grass with a cheerful expression. Adding the breed question to the same prompt made it name the breed too.

Option two: an image on your own machine

The provider cannot fetch a file that only exists on your laptop, so you send the bytes yourself, encoded as base64.

```

import base64
from pathlib import Path

image_path = Path(__file__).parent / "sample.jpg"

encoded = base64.b64encode(
    image_path.read_bytes()
).decode("utf-8")

local_message = HumanMessage(content=[
    {
        "type": "text",
        "text": "What animal is in this photo, and what is it doing?"
    },
    {
        "type": "image",
        "base64": encoded,
        "mime_type": "image/jpeg"
    },
])

print(model.invoke([local_message]).content)

```

This time the answer identified the dog as a Samoyed, sitting on the grass with a happy expression.

WHY BASE64, AND WHERE ELSE YOU MEET IT

An image on disk is bytes. A JSON request body is text. base64 is the standard way of carrying binary data through a text channel, and none of this is AI specific. If you have ever stored an image in a database or embedded one in an email, you have already met the same problem.

Step by step: `read_bytes()` gets the raw file, `b64encode` turns those bytes into base64 bytes, and `.decode("utf-8")` turns those into a string you can put in JSON. `Path(__file__).parent` resolves the folder your script lives in, so the path works regardless of where you launched the terminal.

WHEN TO REACH FOR WHICH

Use the URL form when the image is already public. Use base64 when it is on your machine, behind a login, or produced by your own application, and also when the hosting site blocks server-side downloads, which happens more often than you would expect. base64 makes the request body considerably larger, so it is not free.

6 Prompt templates, and why the user should not write the prompt

The motivation for this one came from a real Telusko product rather than a toy example, and it lands better because of it.

The Alien Coach on the Telusko site generates a personalised course plan. It asks you for a few small things: your background, the tech stack you are interested in, roughly how much experience you have. Then it produces a full plan.

Now think about what had to happen behind that. Everybody knows a detailed prompt produces a better answer. But you cannot require your customers to be good at prompt engineering before they are allowed to use your product. That is not a business.

THE IDEA IN ONE SENTENCE

You write the long, careful prompt once, as the developer. The user supplies two or three small values. The template combines them at runtime. Your user gets an expert-level prompt without ever writing one.

WHAT THE USER GIVES YOU

```
language : Java
limit    : 60
topic    : JDBC
```

WHAT ACTUALLY GOES TO THE MODEL

```
system: "You are a Java trainer.
        Keep answers under 60 words"
human: "Explain JDBC to a beginner."
```

the same template, different values, no rewriting:

```
Python / 80 / decorators → "You are a Python trainer. Keep answers
                           under 80 words" + "Explain decorators
                           to a beginner."
```

```
DevOps / 40 / Kubernetes → ... and so on
```

ONE template. every language, every topic, every length.

prompt-template.py

```
from dotenv import load_dotenv
from langchain_openai import ChatOpenAI
from langchain_core.prompts import ChatPromptTemplate

load_dotenv()

model = ChatOpenAI(
    model="gpt-4o-mini",
    temperature=0
)

prompt = ChatPromptTemplate.from_messages(
    [
        (
            "system",
            "You are a {language} trainer. Keep answers under {limit} words"
        ),
        (
            "human",
            "Explain {topic} to a beginner."
        )
    ]
)

filled = prompt.invoke({
    "language": "Java",
    "limit": 60,
    "topic": "JDBC"
})

print(filled.messages)
print()

response = model.invoke(filled)
print(response.content)
```

Reading it

- `ChatPromptTemplate` comes from `langchain_core.prompts`.
- `from_messages` takes a list of tuples. The first element is the role, `"system"` or `"human"`, and the second is the text.

- Anything in `{curly braces}` is a placeholder. Notice the template never names a language, a topic or a word limit. It is deliberately generic.
- `prompt.invoke({...})` fills the placeholders and returns the finished message list.
- `model.invoke(filled)` then sends that.

PRINT THE FILLED PROMPT BEFORE YOU SEND IT

That `print(filled.messages)` line is worth keeping in every template you write. It shows exactly what is about to go to the model, after substitution. When an answer comes back wrong, this is the first place to look, because the usual cause is that the assembled sentence did not read the way you imagined.

It also caught something live: asking for "Harness Engineering" produced an answer about electrical wiring harnesses, because the template said "Java trainer" and nothing narrowed it to AI. The template was fine, the values were wrong.

7 Output parsers

Everything so far has ended with `response.content`, which is a block of text. That is exactly what an end user wants. It is often not what the next line of your program wants.

Parsing, in general, means converting something into a usable type. Java developers know it as Jackson for objects to JSON, or marshalling and unmarshalling for XML. Python has `json.dumps`. Same idea here, done by LangChain.

```
from langchain_core.output_parsers import (
    StrOutputParser,
    JsonOutputParser
)
```

- `StrOutputParser` takes the `AIMessage` and gives you a plain Python string.
- `JsonOutputParser` takes the `AIMessage` and gives you a Python dictionary.

output-parser.py

```
from dotenv import load_dotenv
from langchain_openai import ChatOpenAI
from langchain_core.output_parsers import StrOutputParser, JsonOutputParser
from langchain_core.prompts import ChatPromptTemplate

load_dotenv()

model = ChatOpenAI(model="gpt-4o-mini", temperature=0)

# 1. AIMessage in, plain string out
text_parser = StrOutputParser()

response = model.invoke("Name three python web frameworks")
parsed_text = text_parser.invoke(response)
print(parsed_text)
print()

# 2. AIMessage in, Python dictionary out
json_parser = JsonOutputParser()

json_prompt = ChatPromptTemplate.from_messages([
    (
        "system",
        "Reply with JSON only, using the keys name, creator and year."
    ),
    (
        "human",
        "Tell me about the {topic} framework."
    ),
])

filled = json_prompt.invoke({"topic": "Django"})
raw_reply = model.invoke(filled)

data = json_parser.invoke(raw_reply)
print(type(data))
print(data["name"], "was created in", data["year"])
```

WHY THE STRING PARSER LOOKS LIKE IT DOES NOTHING

Printed on screen, the output before and after `StrOutputParser` looks identical, and this confused the room. It is not doing nothing. It has changed the object's type from an `AIMessage` to a Python string. Text looks like text either way.

The JSON parser is where the difference becomes visible. `print(type(data))` shows a `dict`, and `data["name"]` works, which it never would on an `AIMessage`. That is the point of parsers: not prettier output, but a usable type for the next line of your code.

PARSERS DO NOT CALL THE MODEL

Asked in class and worth being clear about. By the time a parser runs, the model call has already happened. `model.invoke()` hit the LLM; `parser.invoke()` just converts what came back. No extra tokens, no second request.

STRUCTURED OUTPUT, AND WHY IT WAS POSTPONED

Asking for JSON in the system prompt works most of the time. It is not a guarantee, because the model can add a sentence before it or wrap it in a code fence.

The proper answer is **structured output**, where you describe the shape as a Pydantic model and the provider is made to fill exactly that shape. It was named in class and deliberately deferred, because it needs Pydantic and the Pydantic recordings were still in editing. It arrives once those are on the dashboard.

Reference: docs.langchain.com/oss/python/langchain/structured-output

8 Runnable: the common interface

Somebody noticed something in the chat that turned into a proper explanation. Look at what has been written so far:

```
prompt.invoke({...})    # a prompt template
model.invoke(filled)    # a chat model
parser.invoke(response) # an output parser
```

Three completely different objects, one method name. That is not a coincidence.

RUNNABLE, DEFINED

A **Runnable** is LangChain's common interface. Anything that is a Runnable can be executed with the same set of methods: `invoke`, `batch` and `stream`. The phrasing used in class is the one to remember: *a Runnable is something LangChain knows how to run and connect*.

A prompt is a Runnable. A model is a Runnable. A parser is a Runnable. And, as the next section shows, a chain of them is also a Runnable, which is what makes chains nest inside other chains.

Two consequences worth carrying forward. First, once you know one component, you know how to call all of them. Second, and this is the part that matters later, if everything speaks the same interface, everything can be plugged into everything else.

9 LCEL, the pipe operator

Look at what the output parser example actually did, step by step:

1. `json_prompt.invoke({...})` to fill the template, giving output one
2. `model.invoke(filled)`, passing output one in, giving output two
3. `json_parser.invoke(raw_reply)`, passing output two in, giving the final result

Three invocations, and you carried the result from each one into the next by hand. That is a chain, just a manual one.

LangChain Expression Language lets you declare the same thing with the pipe operator. The comparison drawn was the Linux shell pipe, which does exactly this: the output of one command becomes the input of the next.

BY HAND

```
filled = prompt.invoke({...})
reply  = model.invoke(filled)
result = parser.invoke(reply)
```

you carry each output
into the next call

three statements, three variables

WITH LCEL

```
chain = prompt | model | parser
result = chain.invoke({...})
```

LangChain carries them for you
prompt output → model input
model output → parser input

one statement, no variables

lcel_pipe.py

```
from dotenv import load_dotenv
from langchain_openai import ChatOpenAI
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.output_parsers import StrOutputParser

load_dotenv()

prompt = ChatPromptTemplate.from_messages([
    (
        "system",
        "You are a helpful tutor. Answer in {limit} words or less."
    ),
    (
        "human",
        "{question}"
    ),
])

model = ChatOpenAI(
    model="gpt-4o-mini",
    temperature=0
)

parser = StrOutputParser()

chain = prompt | model | parser

print(chain.invoke({
    "limit": 60,
    "question": "What is an API?"
}))
```

You never wrote `prompt.invoke`, `model.invoke` or `parser.invoke` anywhere. You declared the order and LangChain wrote the plumbing. That is what abstraction means here.

TWO THINGS LCEL DOES NOT DO, BOTH ASKED IN CLASS

It does **not** save tokens. It does **not** change the response in any way. Exactly the same calls happen, in exactly the same order, with exactly the same cost. What you get is shorter, cleaner code, and one composable object instead of three loose steps.

Readability is a fair objection and was raised. A long pipeline on one line is harder to read than three statements. Split it across lines, or keep the manual version when a step needs explaining. Nothing forces you to use the pipe.

10 Chaining a chain into another chain

Here is where the Runnable idea pays off. A chain is itself a Runnable, so it can be piped into another chain.

```
translate = (
    ChatPromptTemplate.from_messages([
        (
            "human",
            "Translate this to Hindi:\n\n{text}"
        )
    ])
    | model
    | parser
)

full = chain | translate

print(
    full.invoke({
        "limit": 30,
        "question": "What is a Java?"
    })
)
```

```

input: { limit: 30, question: "What is Java?" }
      |
      +-----v-----+
      | CHAIN 1          |
      | prompt → model → parser |  answers the question
      +-----+-----+
      |
      | an English answer
      |
      | becomes {text} for the next template
      v
      +-----+-----+
      | CHAIN 2 (translate) |
      | prompt → model → parser |  translates it
      +-----+-----+
      |
      v
      the answer, in Hindi

```

Demonstrated live in both Kannada and Hindi. Two model calls happen, one per chain, and LangChain passes the first chain's output straight into the second chain's `{text}` placeholder without you writing a line for it.

WHY THIS SHAPE MATTERS LATER

This is the pattern behind a great deal of what comes next. Fetch context, then answer from it. Rewrite the question, then search with it. Answer, then check the answer. Each stage is a chain, and chains compose. The RAG pipelines in a later module are this idea with a retrieval step in the middle.

Reference: docs.langchain.com/oss/python/langchain

11 A model from Hugging Face

Back to section 2's second route. The model lives on Hugging Face and runs on their infrastructure, so this is neither the OpenAI path nor the local path.

Getting a token

1. Sign up at huggingface.co.
2. Click your profile, then **Settings**, then **Access Tokens** in the left menu.
3. Create a new token, give it a name, and copy it.

Then add it to the same `.env` file, on its own line beneath the OpenAI key:

```
# .env
OPENAI_API_KEY=sk-your-key-here
HUGGINGFACEHUB_API_TOKEN=hf_your-token-here
```

THE TOKEN GETS YOU IN, IT DOES NOT CHOOSE THE MODEL

Asked directly in class. This token authenticates you to Hugging Face. It does not say which model you want and it does not grant access to every model on the platform. You select the model separately in code, by its repository id, and you have to do that explicitly or nothing works.

WHY THE VARIABLE NAME IS NOT ARBITRARY

A good question came up: can the key be called anything? For your own manual lookups with `os.getenv`, yes. But when you want LangChain to find the credential automatically, the name has to be the one the library expects, which is `HUGGINGFACEHUB_API_TOKEN`. Same reason `OPENAI_API_KEY` is spelled that way. It is a requirement of the library, not a style preference.

hf-demo.py

```
uv add langchain-huggingface
```

```
from dotenv import load_dotenv
from langchain_huggingface import ChatHuggingFace, HuggingFaceEndpoint

load_dotenv()

endpoint = HuggingFaceEndpoint(
    repo_id="meta-llama/Llama-3.1-8B-Instruct",
    task="text generation",
    max_new_tokens=200,
    temperature=0.3,
)

model = ChatHuggingFace(llm=endpoint)

print(model.invoke("Explain what an open weights model is, in two lines").content)
```

Two objects, and the split between them is the thing to understand.

- **HuggingFaceEndpoint** is the connection and configuration. Which model (`repo_id`), what kind of job (`task`), how long the answer may run (`max_new_tokens`), how varied it should be (`temperature`). Many more parameters exist; these were the ones set in class and the rest fall back to defaults.
- **ChatHuggingFace** wraps that endpoint and turns it into a LangChain chat model. Once wrapped, `invoke` works, prompt templates work, parsers work, LCEL works. It behaves like every other model in this course.

Tokens page: huggingface.co/settings/tokens

THREE ROUTES, ONE HABIT

You have now called a model three different ways in three classes: a paid hosted provider, a model on your own laptop, and a model on Hugging Face's servers. In every case the code after the model line was the same. Get into the habit of treating the model as one swappable line, and provider choice becomes a business decision rather than a rewrite.

12 Temperature, explained properly

`temperature` has appeared in several files without explanation. It got one here.

It controls how varied the output is when you ask the same question twice.

<code>temperature = 0.0</code>	maximum consistency ask "what is Java?" twice, get almost the same answer both times. most predictable.
<code>temperature = 0.3</code>	small variation in wording
<code>temperature = 0.8</code>	high creativity and randomness two answers to the same question can read completely differently
pick low for extraction, classification, anything a machine reads next pick high for brainstorming, marketing copy, creative writing	

That is why most of the class files set `temperature=0`. During a demonstration you want the same result twice so you can talk about it. It is also why a JSON-producing prompt should run at zero: you are not looking for creativity, you are looking for a shape.

THIS EXPLAINS A QUESTION FROM THE FOLLOWING DAY

Two students ran the same prompt template and got different outputs, and asked whether temperature caused it. Partly, but not only. Models predict rather than replay stored answers, so identical input can produce different wording even at low temperature. Temperature widens or narrows that spread, it does not eliminate it.

13 Licences, cost and production reality

The session closed on the practical questions, and the answers were more careful than the usual "open means free".

Open weights does not mean unrestricted

Downloading a model does not give you the right to use it in a revenue-generating product. Each model carries a licence, and some restrict commercial use. The parallel offered was ordinary software: plenty of things are free to learn with and licensed differently the moment money is involved.

The licence is stated on that model's own page on Hugging Face. Check it before you build on it, not after.

Running locally is not free either

You are not paying per token, you are paying for GPUs, memory and the electricity to run them. For a serious model that bill is real, it is just capital instead of usage. You also inherit the work: safety, guardrails and everything the hosted provider was doing for you.

Is Hugging Face usable in production?

The room mostly voted no, and the honest answer was that it depends.

- If the model's licence permits commercial use, you do not pay Hugging Face for the licence itself.
- If you run it on their infrastructure through inference endpoints, you pay for that compute once you go past the free allowance.
- If you download it and host it yourself, there is no Hugging Face fee and you pay for your own hardware instead.

The distinction to hold on to: it is not "can I use Hugging Face in production", it is "what does *this specific model's* licence allow, and whose hardware am I running it on".

A FAIR WARNING ABOUT SCOPE

Stated clearly at the end: do not try to master Hugging Face from one demonstration, or Ollama from one integration. Both come back later with proper depth. What this session gives you is the ability to say what each one is, and to integrate a model from either in a few lines. For a third class, that is the right amount.

14 Clarifications from the Q&A

- **Why does `invoke` work on the prompt, the model and the parser?** Because all three are Runnables, and `invoke` is part of that common interface. They do different work, but they are called the same way.
- **Is LCEL a token-saving technique?** No. It is code optimisation only. Same calls, same cost.
- **Does the string output parser do anything visible?** Not on screen. It changes the type. The JSON parser is where you see the difference, because `data["name"]` works on a dictionary and not on an AIMessage.
- **Does the parser hit the LLM?** No. The model call already happened before the parser runs.
- **Does Ollama run continuously on my machine?** Not once the model is unloaded. You can remove a model entirely after experimenting, and performance is unaffected.
- **Does Ollama need internet?** Only to pull the model the first time. After that it runs offline. Hugging Face endpoints always need the network, because the model runs on their servers.
- **Why Mistral for the local demo?** Small, free, easy to run locally, no vendor lock-in, good enough for experimentation. Fewer parameters than a frontier model, which is the trade.
- **What laptop configuration do I need?** The instructor's machine has 24 GB of RAM. Requirements scale with the model you choose, and the topic gets proper treatment later.
- **Can I fine tune these?** Closed models usually do not permit it. Open weights do. Fine tuning is on the syllabus, though it will be demonstrated rather than practised by everyone, because it needs hardware.
- **Which model powers the Telusko Alien Coach?** Several were tried, including Google's Gemma and a recent Meta model, with different results from each. It gets discussed properly later.
- **Can the notes be grouped by session?** A student asked and the suggestion was accepted.
- **LLM engineer against AI engineer?** Roughly: building and fine tuning the models themselves is one job. Building applications on top of existing models, with tools, RAG and the rest, is AI engineering, and that is what this course teaches.
- **Where do notes and code go?** Course dashboard, under the LangChain live classes section, in the resources for each session: notes, board work and source code, plus a GitHub link.

15 Key takeaways

- **Hugging Face is GitHub for trained models.** Text, image, speech, embeddings, all discoverable in one place.

- **Closed model against open weights is a real decision:** per-token cost and your data leaving, versus hardware cost and your data staying.
- **Open weights is not open source.** You get the trained weights, not the training code or data, and licences still apply.
- **Ollama runs a model on your own machine.** No key, no `load_dotenv()`, no per-call cost, no internet after the pull.
- A `HumanMessage` can carry blocks, so text and an image travel in one message. URL for public images, base64 for local ones.
- **Prompt templates let you write the good prompt so your user does not have to.** Print the filled prompt while you are developing.
- **Output parsers convert the reply into a usable type,** a string or a dictionary, without another model call.
- **Everything is a Runnable,** which is why `invoke` works everywhere and why pieces connect.
- **LCEL replaces three manual invocations with `prompt | model | parser`.** Cleaner code, identical behaviour and cost.
- **A chain is a Runnable too,** so chains nest. That is the shape most later pipelines take.
- `HuggingFaceEndpoint` plus `ChatHuggingFace` turns a hosted open model into an ordinary LangChain chat model.
- **Temperature is consistency against creativity.** Zero for anything a machine reads next.
- **The model is one swappable line.** Everything after it stays the same across OpenAI, Ollama and Hugging Face.

COMING NEXT

The very next day. Streaming, so the answer appears piece by piece instead of all at once. Image generation, the reverse of section 5. Then the topic everything after this depends on: **tool calling**, also called function calling, because a tool is just a function. It was flagged as essential, on the grounds that MCP later will be difficult without it. Structured output with Pydantic follows once the Pydantic recordings are published.